Latest updates: https://dl.acm.org/doi/10.1145/3583133.3596944

ABSTRACT

Tree-Based Grammatical Evolution with Non-Encoding Nodes

MARINA DE LA CRUZ LÓPEZ, Complutense University of Madrid, Madrid, Madrid, Spain

OSCAR GARNICA, Complutense University of Madrid, Madrid, Madrid, Spain

J. IGNACIO HIDALGO, Complutense University of Madrid, Madrid, Madrid, Spain

Open Access Support provided by:

Complutense University of Madrid

PDF Download
3583133.3596944.pdf
26 February 2026
Total Citations: 0
Total Downloads: 221

Published: 24 July 2023

Citation in BibTeX format

GECCO '23 Companion: Companion Conference on Genetic and Evolutionary Computation
July 15 - 19, 2023
Lisbon, Portugal

Conference Sponsors:
SIGEVO

Tree-Based Grammatical Evolution with Non-Encoding Nodes

Marina de la Cruz López
Universidad Complutense de Madrid
Madrid, Spain
marcru06@ucm.es

Oscar Garnica
Universidad Complutense de Madrid
Madrid, Spain
ogarnica@ucm.es

J. Ignacio Hidalgo
Universidad Complutense de Madrid
Madrid, Spain
hidalgo@ucm.es

ABSTRACT

Grammar-guided genetic programming is a type of genetic programming that uses a grammar to restrict the solutions in the exploration of the search space. Different representations of grammar-guided genetic programming exist, each with specific properties that affect how the evolutionary process is developed. We propose a new representation that uses a tree structure with non-encoding nodes for the individuals in the population, a.k.a. Tree-Based Grammatical Evolution with Non-Encoding Nodes. Each tree's node has a set of children nodes and an associated number that determines which are used in decoding the solution and which are non-encoding nodes. This representation increases the size and complexity of the individuals while performing a more exhaustive exploration of the solution space. We compare the performance of our proposal with state-of-the-art genetic programming algorithms for the 11-multiplexer benchmark, showing encouraging results.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; **Search methodologies**;

KEYWORDS

Grammars, Genetic Programming, Grammar-Based Genetic Programming

ACM Reference Format:
Marina de la Cruz López, Oscar Garnica, and J. Ignacio Hidalgo. 2023. Tree-Based Grammatical Evolution with Non-Encoding Nodes. In *Genetic and Evolutionary Computation Conference Companion (GECCO '23 Companion), July 15–19, 2023, Lisbon, Portugal.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3583133.3596944

1 DESCRIPTION

Grammar-Guided Genetic Programming (GGGP) is a type of genetic programming (GP) that uses a grammar to determine the representation structure of the individuals in the population. The use of a grammar in the construction of the solutions allows (i) to create individuals that contain expert knowledge for the specific problem, and (ii) to limit the accessible search space.

Different GGGP's versions include Grammatical Evolution (GE) [1], which uses a linear representation of the genotype; Structured Grammatical Evolution (SGE) [2], which uses multiple linear representations, one for each rule of the grammar; and, Context-Free Grammar GP (CFG-GP) [3], where the genotype is a derivation tree generated from the grammar. Each representation has a different effect on the evolutionary process, with its advantages and drawbacks.

We propose a novel form of GGGP that combines the features of CFG-GP and GE, namely Tree-Based Grammatical Evolution with Non-Encoding Nodes (T-GE-NEN). Our proposal's main novelty is using non-encoding nodes, i.e., nodes that have no immediate effect on the phenotype but carry out a parallel evolutionary process that may manifest on the phenotype after eventual mutation or crossover processes. These nodes preserve extra information of the grammar structure so that the evolutionary operators are not as destructive as other GGGP versions.

T-GE-NEN uses a tree representation in which each node represents a non-terminal symbol of the grammar. Each node contains three attributes: its non-terminal symbol that determines the rule associated with the node, a number that selects the production of the rule, and the set of all possible children nodes for the following non-terminal symbols that the node could generate.

Compared to a derivation tree, this set may contain both codifying and non-codifying nodes. Whether a node is codifying or not is determined by the production generated by its parent node. Figure 1 illustrates a portion of a tree. The upper node represents the $< expr >$ non-terminal symbol, and the number 1 selects the second production in the RHS of the rule associated with the non-terminal symbol yielding to $< expr >::=< expr >$. Dividing this production into its non-terminal elements just yields one non-terminal $< expr >$. This non-terminal has a children node associated with the same symbol. This example is highlighted using blue background and solid lines in Figure 1. If, during evolution, the number associated with the parent node changed (e.g., from 1 to 0), the production would now be $< expr >::=< expr >< op >< expr >$. The new production contains three non-terminal symbols. When decoding the individual, T-GE-NEN would take three children nodes: the first node of type $< expr >$, which is the same node as the previous production, the node of type $< op >$, and the second node of type $< expr >$. This configuration appears in red hatch in Figure 1. One of T-GE-NEN's advantages is that the nodes are shared among productions when they have the same type and amount non-terminals.

The set of children for each node is generated dynamically. Initially, each node will only generate the set of nodes that composes the production chosen for the node. As the evolution process advances, if a production cannot be decoded (because it does not exist in the structure), T-GE-NEN generates the sub-tree to be able to complete the individual. This is a different approach to

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GECCO '23 Companion, July 15–19, 2023, Lisbon, Portugal
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0120-7/23/07.
https://doi.org/10.1145/3583133.3596944

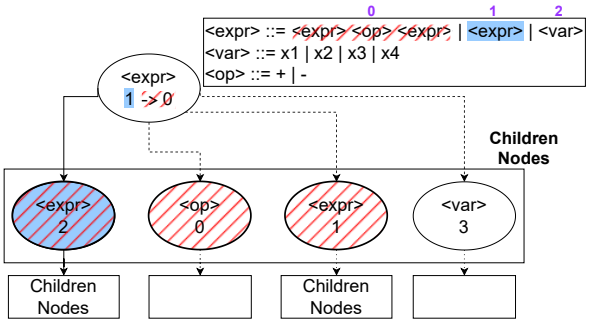

Figure 1: T-GE-NEN example

traditional tree representation, where evolution is produced on two levels: encoding and non-encoding.

The crossover operator is the general sub-tree operator but is restricted only to the codifying nodes. It selects two codifying nodes of two individuals with the same non-terminal symbol and interchanges the sub-trees.

The mutation is performed on the numbers associated with each node. It replaces this number with a random one between 0 and the total number of productions for the specific non-terminal symbol. As with the crossover, the mutation is only performed over the codifying nodes. In addition, we stop the mutation process on a sub-tree once we have mutated the root of said sub-tree since mutating the root will change the structure underneath.

During evolution individuals suffer two transformations: the crossover modifies the tree structure (the number of nodes and its edges) whereas the mutation modifies one of the attributes of the node, i.e., the number associated with the node. The latter modification can collaterally alter the tree structure as well, creating new nodes in case they did not exist.

2 PRELIMINARY RESULTS

We compare the efficiency of GE, DSGE, CFG-GP, and T-GE-NEN for the 11-bit multiplexer benchmark. The benchmark's goal is to approximate the performance of an 8-bit multiplexer with a 3-bit select signal. Table 1 resumes the experimental setup for the comparison, and Figure 2 presents the grammar.

```
<start> ::= ( <B> ? 1 :0 )
<B> ::= <B> "&" <B> | <B> "|" <B> | !(<B>) | !(<B> "|" <B>)
      | (<B> ? <B> : <B>) | toBool(<var>)
<var> ::= x1 | x2 | x3 | d0 | d1 | d2 | d3 | d4 | d5
      | d6 | d7
```

Figure 2: 11-bit multiplexer grammar

Figure 3 shows the evolution of the best individual for 1000 generations in 30 runs. The solid line represents the average fitness value of the best individual in each generation, and the error band is the standard deviation of the best fitness per generation. The fitness metric considered is the absolute error.

Our proposal performs better or equal to the GGGP alternatives regarding the final fitness value in training. T-GE-NEN starts the evolutionary process very similarly to CFG-GP and is better than both DSGE and GE. CFG-GP and T-GE-NEN perform very similarly until generation 150, when T-GE-NEN starts to give better average results diverging from the line of CFG-GP.

Table 1: Experimental setup for the comparison of GE, DSGE, CFG-GP, and T-GE-NEN. Sub-tree(*) and Basic(*) are T-GE-NEN's crossover and mutation in Section 1

	GE	DSGE	CFG-GP	T-GE-NEN
Runs		30		
Gena		1000		
Pop		200		
Init. Pop.		Random		
Selection		Tournament pressure 2		
Elitism		2%		
Crossover	Single Point	Uniform 0.25 chng	Sub-Tree	Sub-tree(*)
Cros. Prob	0.7	0.7	0.7	0.7
Mutation	Basic	Basic	Sub-tree regeneration	Basic(*)
Mut. Prob	0.02	0.02	0.4	0.01
Depth	-	5	5	5
Chr. length	200 (3 Wraps)	-	-	-

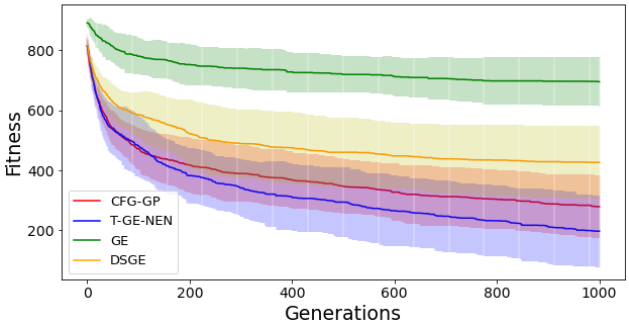

Figure 3: Evolution of the fitness for the best individual in the population for the 11-bit multiplexer benchmark

The fitness difference keeps increasing until generation 800 and then it remains constant till the end. T-GE-NEN's improvement can be due to the non-encoding nodes aiding the evolutionary process once they have developed. For this reason, the fitness improvement happens after generation 150. As future work, we will study the efficiency of this algorithm with other benchmarks, as well as with different versions of the evolutionary operators.

ACKNOWLEDGMENTS

This work is supported by Spanish Government MINECO grants PID2021-125549OB-I00 and PDC2022-133429-I00.

REFERENCES

[1] Ian Dempsey, Michael O'Neill, and Anthony Brabazon. 2009. *Grammatical Evolution.* Springer Berlin Heidelberg, Berlin, Heidelberg, 9–24. https://doi.org/10.1007/978-3-642-00314-1_2
[2] Nuno Lourenço, Filipe Assunção, Francisco Pereira, Ernesto Costa, and Penousal Machado. 2018. *Structured grammatical evolution: A dynamic approach.* 137–161. https://doi.org/10.1007/978-3-319-78717-6_6
[3] P.A. Whigham. 1999. Grammatically-based Genetic Programming. (06 1999).